

LINE THICKENING BY MODIFICATION TO BRESENHAM'S ALGORITHM

A. S. Murphy

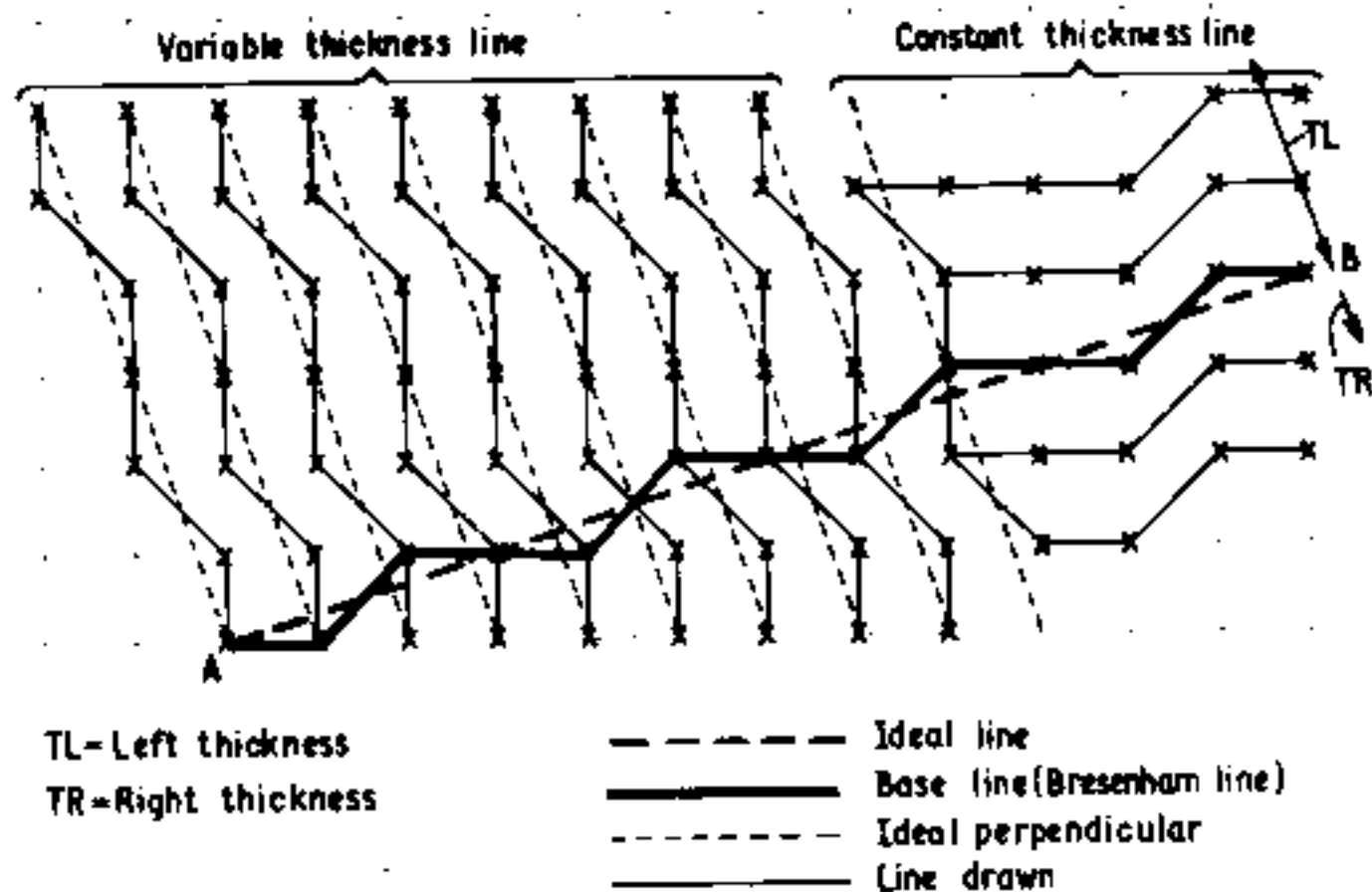


FIG. 1

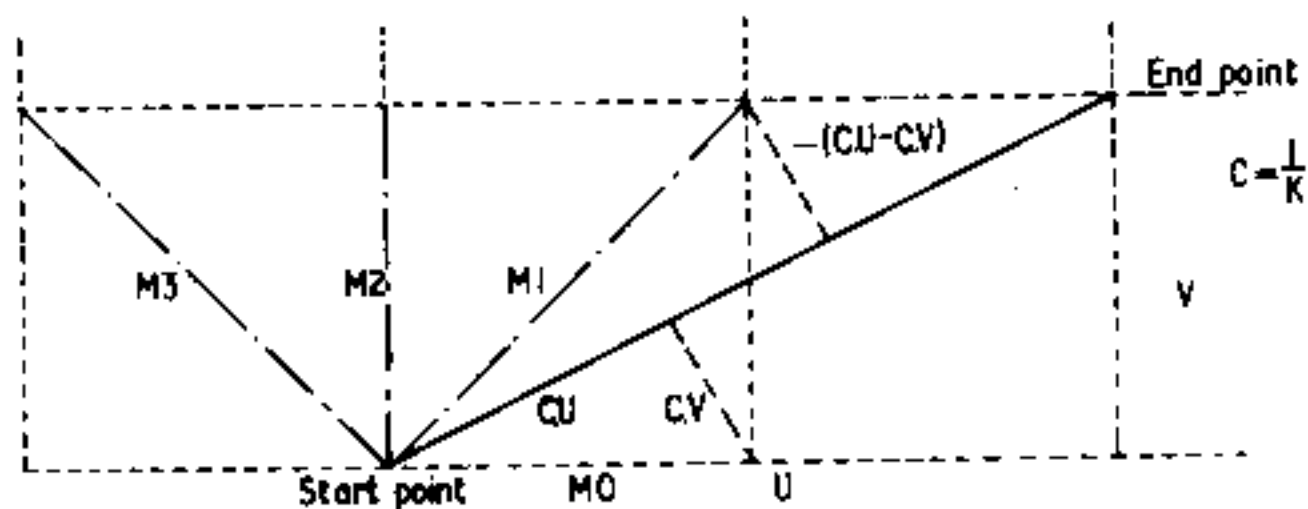
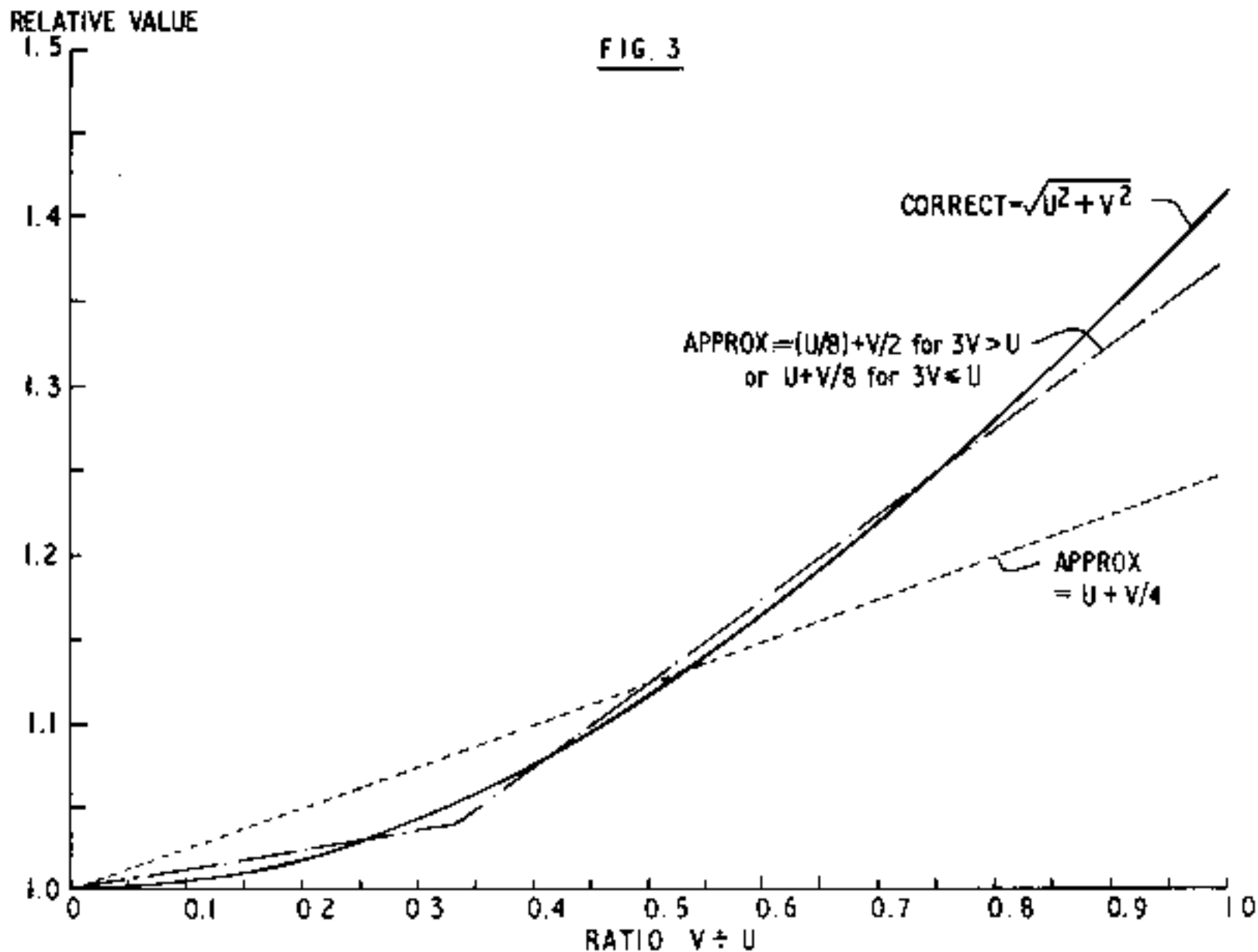


FIG. 2

This article describes an algorithm for producing thickened lines on a display or picture grid which may be used to generate intermediate points for control of an incremental plotter or display device or for storage in a raster bit image. Modifications to the algorithm may be

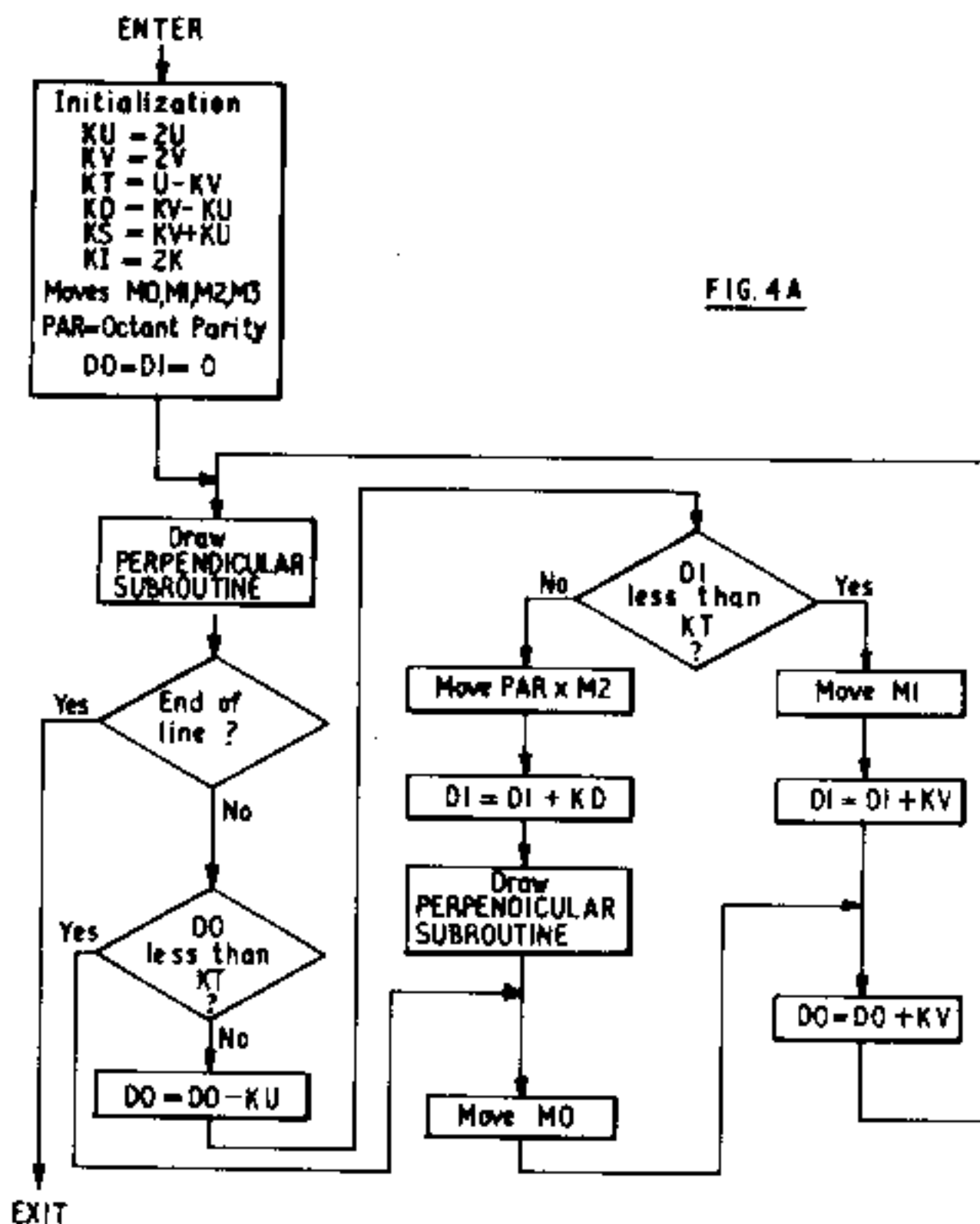


used to generate lines of constant or variable thickness and to produce shaded area images.

Previous line thickening algorithms suffer from several problems. The same point may be generated more than once, resulting in inefficiency and poor performance: images resulting from such an algorithm cannot be exclusive-ORed with a bit image, as a repeatedly generated point will produce an indeterminate result. Line thickness is specified as a constant, and this may be measured only in the two orthogonal directions: this gives a variation in line thickness by up to a factor of 1.414 between orthogonal and diagonal lines. Because line thickness is defined as a fixed number of picture elements (pels), lines of varying thickness cannot be drawn. The end effects vary according to line orientation which may produce diamond-shaped ends for orthogonal lines and square-shaped ends for diagonal lines. The end effects can cause merging of the line structure for structured lines, for example, dotted lines. Finally, some algorithms cannot be applied successfully to curved lines.

The main advantages of the algorithm described below are:

- a) In the inner loop, the only operations required are addition



and comparison testing, which means a simple computing engine can be used.

- b) For straight lines, each point is generated only once.
- c) The line thickness need not be an integral number of pels.
- d) The line thickness can be made a function of line length or derived from Z-axis data and can be used for depth-cueing.
- e) The perceived thickness does not vary for lines at different orientations.

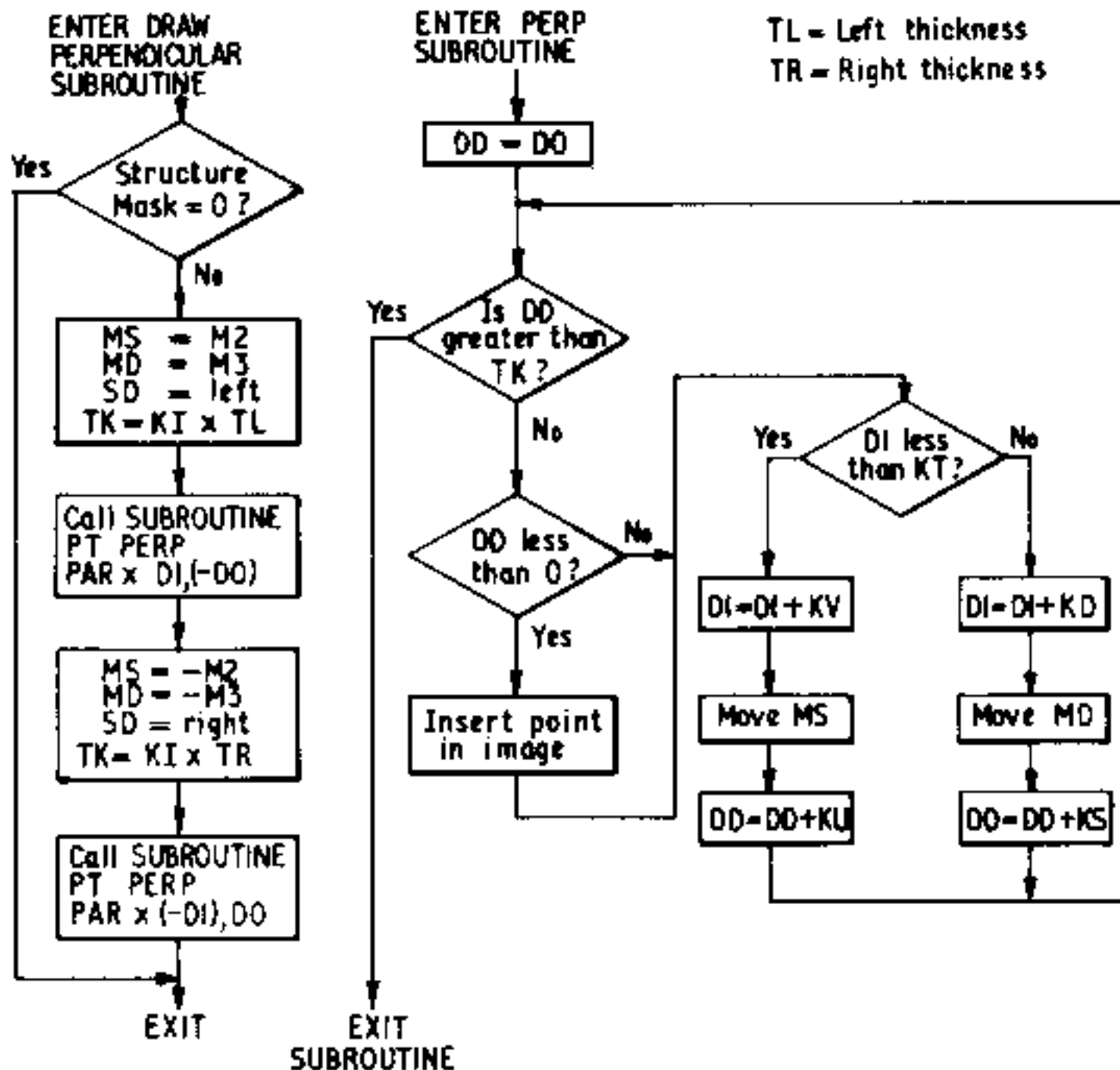
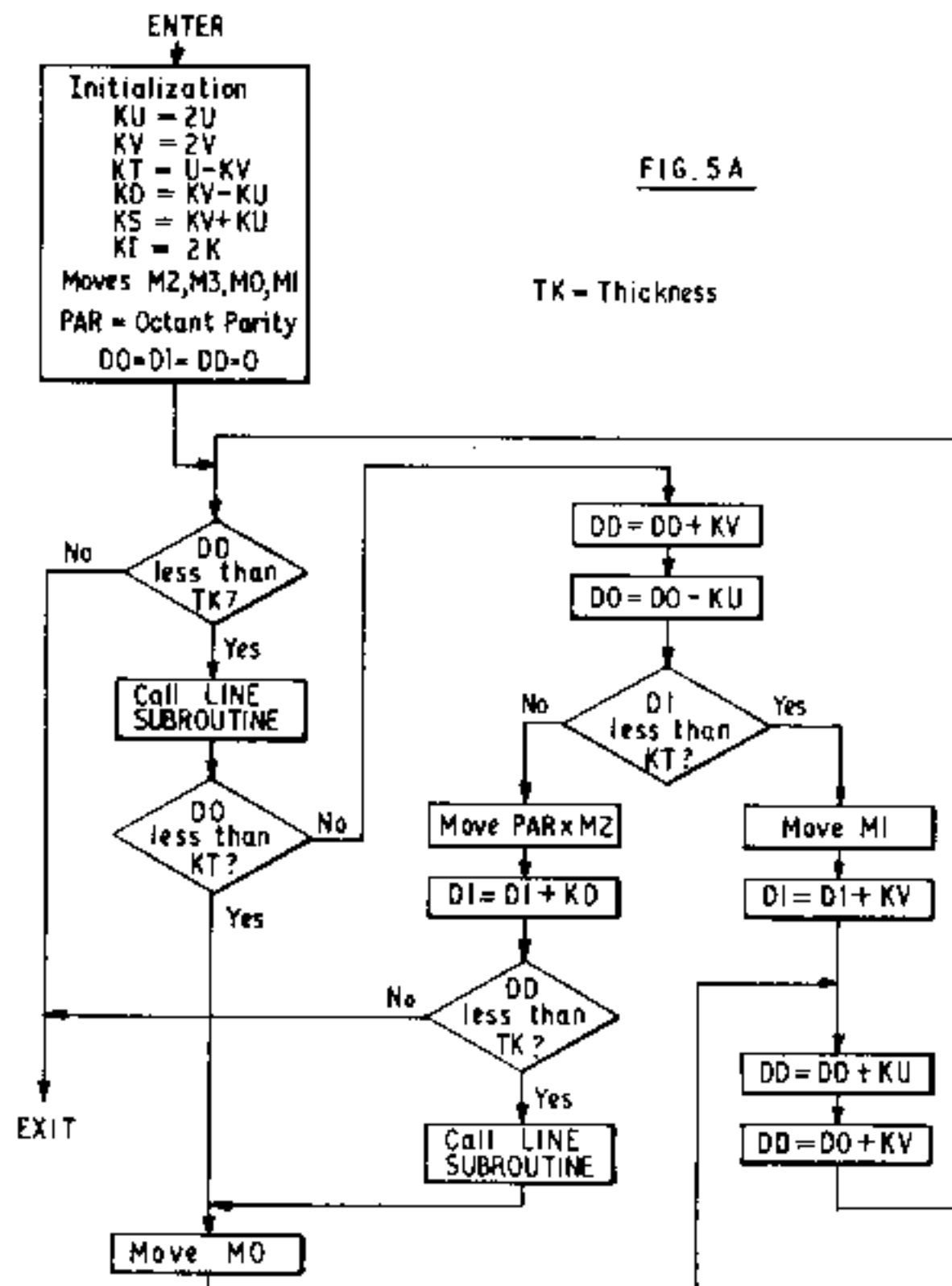


FIG. 4B

- f) There are no unsightly end effects.
- g) The thickness can be defined as a complex function and the resulting image generated as a graph or histogram.
- h) The algorithm can be used to generate area-filled or shaded objects, such as squares, hexagons, diamonds and circles.

Referring to Fig. 1, the requirement is to generate all points which are within a distance T_L to the left of a line AB and within a distance T_R to the right of the line AB. For normal constant-thickness lines, $T_L = T_R = \text{a constant}$. However, in the general case,



$T_L = F_l(L) =$ a function of line length (L), and

$T_R = F_r(L) =$ another function of line length (L).

The algorithm is an extension of the line drawing algorithm originally proposed by J. E. Bresenham in IBM Systems Journal 4, 25-30 (1965), and subsequently developed by M. L. V Pitteway in Computer Journal 10, 282-289 (1967), by D. Cohen in Computer Graphics 70, Section 8 (International Symposium, Brunel University 1970), and B. K. P. Horn in Computer Graphics and Image Processing 5, 280-288 (1976).

The basic straight-line Bresenham algorithm produces the closest

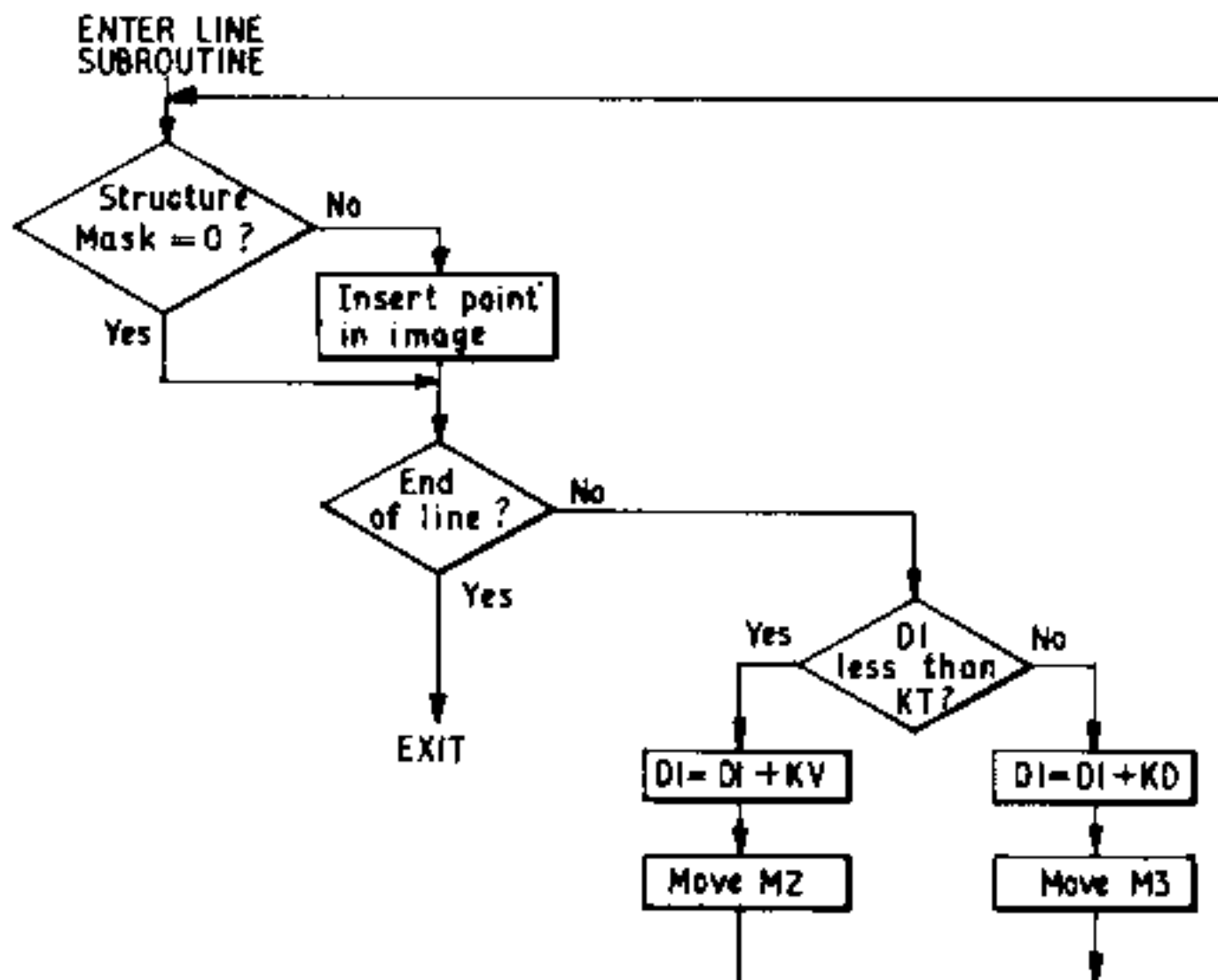


FIG. 5 B

grid points to a given line. As shown in Fig. 1, all points lie within a half pel of the ideal line: the set of generated points is called the base line. In order to produce a thickened line, all points within a specified distance of the ideal line must be generated. This is accomplished by modifying the basic line algorithm so that at each base line point, a perpendicular line of points extending a distance T_L or T_R from the ideal line is also generated. The technique is analogous to a recursive call of the line drawing algorithm from within itself. However, because the terminating conditions are different for the two levels, it is more practical to generate the perpendicular points using a separate subroutine. Perpendicular lines may be drawn separately on both the left and right sides of the ideal line using different thickness functions.

The key aspect of the technique is that the inner loop of the basic line algorithm contains just the right information for a perpendicular line which passes through equally spaced points on one of the axes. Referring to Fig. 2, consider a line starting at (0,0) and ending at (U,V) when $U \geq V$. Let M0, M1, M2 and M3 be four moves such that the change in position on the coordinate grid is given by the following table:

LINE THICKENING BY MODIFICATION TO BRESENHAM'S ALGORITHM - Continued

<u>Move</u>	<u>Dx, Dy</u>
M0	1 0
M1	1 1
M2	0 1
M3	-1 1

If D0 is 2K times the perpendicular distance of a grid point from the line and D1 is 2K times the parallel distance along the line, where K is a normalization constant which maintains the parameters as integral values, then the effect upon D0 and D1 for each of the different moves is:

<u>Move</u>	<u>Change to D0</u>	<u>Change to D1</u>
M0	KV	KU
M1	KV-KU	KU+KV
M2	-KU	KV
M3	-(KV+KU)	KV-KU

D0 is used to control the move selection and D1 to control the line thickness. It is convenient to initialize the inner loop parameters in a set-up routine:

```

KU = 2 * U
KV = 2 * V
KD = KV + KU
KS = KV - KU
KT = U - KV
KI = 2 * K

```

The moves M0, M1, M2, M3 are set up according to the octant in which the line is drawn. An extra constant PAR is set up to indicate the octant parity (odd octant = -1, even octant = +1).

The normalization constant K is used only to relate the specified thickness to the loop control parameters. Its correct value is $U^2 + V^2$. This is calculated in the initialization routine. In some machines it may prove difficult to compute this value exactly, but because there is a restricted domain for the values of U and V (V/U is between 0 and 1), then it is possible to define an approximation over the limited domain of the function. Thus the expression $U + V/4$ has a maximum error for the correct value of 12%, and this may be acceptable for many applications since the error only relates to a variation in line thickness for lines at different orientations. A more accurate approximation is:

```

If (V + V + V) > U then approximation = U - (U/8) + V/2
else approximation = U + V/8.

```

This approximation has a maximum error of 2.7%, and yet it only uses addition, comparison and right shifting operations. Such an error

should be acceptable for all except the most critical applications. Fig. 3 shows the relationship between the correct value and these two approximations.

The moves required for the perpendicular line (M2,M3) are simply the base line moves (M0, M1) rotated by $+\pi/2$, respectively. The line parameters are identical, and the initial difference term is easily obtained from the base line term. The terminating parameter can be obtained from the base line parameters, or it may be set up during the initialization procedure.

Because there is a requirement to generate each point only once, the perpendiculars should be "in phase" with each other, and this is achieved by ensuring that the perpendicular differences D1 are all identical at grid points which lie parallel to an axis. The use of this technique for initializing the perpendicular differences means that there are some cases when two perpendicular lines must be drawn for one base line point move. This occurs whenever $D0 > KT$ and $D1 > KT$, that is, whenever a diagonal base line move is coincident with a diagonal perpendicular move. The full flow chart is shown in Figs. 4A and 4B.

If lines of varying thickness are not required, the algorithm can be simplified by removing the thickness test from the inner loop and placing it in the controlling loop. Thus, the base line can be drawn using the standard line algorithm as the inner subroutine. The start point is moved to a new start and another parallel line drawn. This is repeated, the new start point being generated along a perpendicular until the required thickness is achieved. An example of a thickened line is shown in Fig. 1. The simplified flow charts are shown in Figs. 5A and 5B.

The application of the algorithm to curved lines means that the values KU , KV , etc., are no longer constant, but change as the curve is traversed. These values are updated during curve generation as follows:

For orthogonal moves,

$$\begin{aligned} KV &= KV - K1 \\ KD &= KD + K2 \end{aligned}$$

For diagonal moves,

$$\begin{aligned} KV &= KV - K4 \\ KD &= KD + K3 \end{aligned}$$

Where $K1$, $K2$, $K3$ and $K4$ are the curvature constants. The thick line algorithms will then draw lines perpendicular to the curve at all points along the curve.

If a transverse line pattern is added to the thick line algorithms described above, various effects can be obtained. A normal line pattern

is applied parallel to the line direction to produce, for example, dotted or dashed lines. If a secondary line pattern is applied perpendicular to the line, then a variety of effects can be obtained. A transverse dot pattern will produce sets of parallel lines. The two edges of a thickened line can be drawn by setting an appropriate transverse line pattern.

The combination of both transverse and normal line patterns using a logical function yields several results. The exclusive-OR function produces a checkered line pattern which may be seen as a thick grey line (half-density). The OR function produces a three-quarter density line and the AND function a quarter-density line. The OR of a two-parallel-line transverse pattern with a dashed longitudinal pattern produces an effect similar to "railway lines".

If the coordinate address is used as the pattern control index rather than line counters, then a variety of other effects can be obtained. In this case the line patterning will always be orthogonal to the picture coordinate system instead of being orthogonal to the line. Clearly, a large number of graphic effects can be obtained by using a combination of two orthogonal line patterns.